🏗️ StickerNest V5 — Master Build Plan

A complete, ground-up plan for StickerNest V5. Designed to be executed with parallel AI agents (Claude Code), proper testing gates, and clean architectural layers. Every decision here should be traceable to the Terminology doc.

> **Source of truth**: Refer to 📚 StickerNest Terminology & Definitions before naming anything in code.

Conceptual Model vs Build Order

These are two different things. Do not confuse them.

Conceptual model (the architecture — how to think about it):

Layer	What it is
Layer A: Spatial	Canvas, entities, rendering, input
Layer B: Data	DataSources — persistent records independent of widgets
Layer C: Interaction	Widgets, Scripts, Pipelines, Event Bus

Build order (the implementation sequence — how to build it):

Layer 0 (Kernel) → Layer 1 (Social) → Layer 3 (Runtime) → Layer 2 (Lab) → Layer 4A/4B (Canvas) → Layer 5 (Marketplace) → Layer 6 (Shell)

The build order exists because of technical dependencies. The conceptual model exists because of clarity. Agents should hold both simultaneously.

> **Source of truth**: Refer to 📚 StickerNest Terminology & Definitions before naming anything in code.

Architecture Overview

StickerNest V5 is built in 7 ordered layers. Each layer is independently testable before the next begins. Agents should never skip ahead.

```
Layer 6: PLATFORM SHELL       Canvas Gallery, auth UI, settin
gs, billing
Layer 5: MARKETPLACE          Widget store, publishing, reven
ue, ratings
Layer 4B: SPATIAL / VR        Three.js renderer, WebXR, Quest
3, hand tracking
Layer 4A: CANVAS EDITOR       Split into four sub-layers:
  ├─ 4A-1: Canvas Core        Rendering, pan/zoom, selection,
transforms, input
  ├─ 4A-2: Tools & Content    Sticker/entity tools, hierarch
y, canvas settings
  ├─ 4A-3: Pipelines & Wiring Pipeline graph UI, AI command b
ar, terminal, scripts
  └─ 4A-4: Panels & Containers Panel system, Docker, workspac
e chrome
Layer 3: WIDGET RUNTIME       iframe sandbox, bridge protoco
l, Widget SDK
Layer 2: WIDGET LAB           AI codegen, live preview, node
graph, publish pipeline
Layer 1: SOCIAL + SYNC        Supabase Realtime, presence, co
nflict resolution
Layer 0: KERNEL               Event bus, Zustand stores, type
s, auth, Supabase client
```

Layers 4A and 4B are parallel tracks that share the same entity/hierarchy system underneath.

Layer 4A Sub-Layer Rationale: V4's canvas editor was a monolith responsible for rendering, tools, pipeline wiring, panel management, AI interaction, and Docker — all in one layer. V5 splits these into four sub-layers with clear boundaries. Each sub-layer is independently testable, can be built in sequence, and has a single responsibility. The sub-layers share the same canvasStore and event bus but do not reach into each other's internals.

Zustand Store Architecture

V4 had 11+ scattered stores. V5 consolidates into 7 domain stores. No store should reach into another store's state directly — cross-store communication goes through the event bus or explicit selectors.

`authStore` — user session, JWT, plan tier, permissions

`workspaceStore` — list of canvases, active canvas ID, canvas metadata, slugs, sharing settings

`canvasStore` — entity hierarchy for the active canvas, transform data, z-order. This store owns entity CRUD and spatial relationships only. It does not own selection, history, or UI state.

`historyStore` — undo/redo stack, snapshot management, dirty-state tracking. Extracted from canvasStore because undo/redo is complex enough to be its own module (action grouping, history branching, selective undo). Listens to canvasStore changes via the event bus and records reversible operations independently.

`widgetStore` — widget registry (installed widgets), active widget instances, widget manifests, dock state

`socialStore` — presence map (who is on this canvas), cursor positions, live edit locks, chat messages

`uiStore` — panel visibility, active mode (2D/3D/VR), tool selection, modal state, sidebar state, **selection state** (moved here from canvasStore because selection is a UI concern — which entities are highlighted — not a data concern)

Supabase

- [ ] Schema: `users`, `canvases`, `entities`, `widgets`, `stickers`, `pipelines`, `widget_connections`, `presence`
- [ ] Canvas slugs: unique constraint per owner, slug validation (alphanumeric + hyphens)
- [ ] Row Level Security policies for all tables
- [ ] Auth: email/password + OAuth (Google, GitHub)

- [ ] Storage buckets: widget assets, sticker assets, canvas thumbnails
- [ ] Seed file with 3 test users, 2 canvases, sample widgets

Storage Bucket Architecture

- [ ] Create Supabase Storage buckets: `media/images`, `media/gifs`, `media/video`, `media/audio`, `media/3d`, `stickers`, `widgets`, `canvas-thumbnails`, `user-avatars`, `exports`
- [ ] RLS on all buckets: user-uploaded assets readable by owner only by default; public assets (published widgets, sticker packs) readable by all
- [ ] Widget bucket path convention: `widgets/{widgetId}/{version}/index.html` — versioned, immutable once written
- [ ] Draft widget path convention: `widgets/drafts/{userId}/{draftId}/index.html` — mutable, not CDN-cached
- [ ] Asset upload service: validates file type, enforces size limits, writes to correct bucket, creates File DataSource record simultaneously
- [ ] Size limits enforced at upload: images 5MB, GIFs 15MB, video 50MB, audio 20MB, 3D models 50MB
- [ ] CDN configuration: all public buckets served via Cloudflare CDN with appropriate cache headers
- [ ] Image optimization pipeline: auto-convert uploaded images to WebP, generate thumbnails at 3 sizes
- [ ] GIF optimization: compress on upload, generate WebM version for browsers that support it
- [ ] `user_installed_widgets` table: `(userId, widgetId, pinnedVersion?, installedAt)` — install = one row, not a copy
- [ ] Widget bundle never copied per user — all users reference the same CDN-served file
- [ ] Asset deletion: deleting a File DataSource removes the database record; bucket file deleted only if no other DataSource references it (reference counting)
- [ ] Signed URLs: private assets (exports, user media) served via time-limited signed Supabase Storage URLs

- [ ] Test: upload image → File DataSource created → asset appears in Library Media section
- [ ] Test: install widget → one row in `user_installed_widgets` → widget appears in Library Widgets section

DataSource System (Layer 0 — Kernel)

DataSources are built as part of Layer 0 because everything else depends on them. Widgets, integrations, folder/doc widgets, and the persistence model all reference DataSources.

- [ ] Define `DataSource` base type: `id`, `type`, `ownerId`, `scope`, `canvasId?`, `schema?`, `metadata`, `createdAt`, `updatedAt`
- [ ] Define `DataSourceType` enum: `doc | table | note | folder | file | custom`
- [ ] Define `DataSourceScope` enum: `canvas | user | shared | public`
- [ ] Define `DataSourceACL` type: per-user permissions independent of canvas roles (owner, editor, commenter, viewer)
- [ ] Supabase: `data_sources` table with RLS enforcing ACL rules
- [ ] Supabase: `data_source_acl` table (dataSourceId, userId, role)
- [ ] DataSource CRUD API: create, read, update, delete, list (scoped by user + canvas)
- [ ] Scope resolution: canvas-scoped sources deleted with canvas; user-scoped sources persist across canvases
- [ ] Widget → DataSource reference: `widget_instances` table gains `dataSourceId?` foreign key
- [ ] Multiple widget instances can reference the same DataSource — all stay in sync via Realtime
- [ ] DataSource deletion handling: widgets referencing a deleted source show disconnected state gracefully
- [ ] Integration mapping: external sources (Notion, Google Sheets, Airtable) sync into DataSources of type `table` or `custom`

- [ ] DataSource permission enforcement: widget cannot read/write a DataSource beyond its ACL, even if canvas role would allow it
- [ ] DataSource type implementations:
  - [ ] `doc` — rich text content (used by Doc widget)
  - [ ] `table` — rows + columns with optional schema (used by Table widget, integration layer)
  - [ ] `note` — plain text / markdown (lightweight doc)
  - [ ] `folder` — ordered list of DataSource references (used by Folder widget)
  - [ ] `file` — binary asset reference (image, audio, video, PDF stored in Supabase Storage)
  - [ ] `custom` — arbitrary JSON with user-defined schema
- [ ] DataSource SDK: `StickerNest.dataSource.get(id)`, `StickerNest.dataSource.update(id, data)`, `StickerNest.dataSource.subscribe(id, handler)` (real-time updates)
- [ ] DataSource changes emit `datasource.updated` events on the bus — widgets can react via pipelines
- [ ] Test: two widget instances referencing same DataSource both update when source changes
- [ ] Test: widget cannot write to a DataSource where ACL is viewer-only

---

Layer 1: Social + Sync

- [ ] Supabase Realtime channel per canvas (channel = `canvas:{canvasId}`)
- [ ] Broadcast: cursor positions (throttled to 30fps)
- [ ] Broadcast: entity transforms during drag (optimistic, reconcile on drop)
- [ ] Presence tracking: join/leave events, presence map in `socialStore`
- [ ] **Conflict resolution by data type** (four separate strategies, do not mix them):
  - [ ] **Canvas entities** — last-write-wins for all position, transform, and property changes. No prompt, silent resolution. Presence cursors prevent most real collisions.

- [ ] **Doc DataSources** — Yjs CRDT. Integrate Yjs with the Doc widget's TipTap/ProseMirror editor. All keystrokes from all users merge without data loss.
- [ ] **Table/Custom DataSources** — revision-based conflict detection with LWW fallback. Each row carries a `revision` counter. A client write must include the last-seen `revision`.
  - If the server row revision has advanced since the client last read it, reject the write (409 conflict) and require the client to re-fetch + retry.
  - If the platform cannot safely determine revision order (e.g., near-simultaneous writes / clock skew), apply LWW fallback.
  - UI behavior: if a write is rejected due to stale revision, show a non-intrusive toast ("Row changed — refreshed") and refresh the row before retrying.
- [ ] **Note DataSources** — last-write-wins, no indicator needed.

- [ ] Yjs integration: Y.Doc per `doc` DataSource instance, synced via Supabase Realtime as Yjs update messages

> **Note**: Terminology is the source of truth for conflict strategy. Table/Custom uses revision-gated writes (not cell badges), and Docs use Yjs CRDT.

- [ ] Permission enforcement: check canvas sharing settings before allowing writes
- [ ] Edit lock: optional per-entity lock so two users don't edit same entity simultaneously
- [ ] Social event bus integration: realtime events enter the local event bus as `social.*` events
- [ ] Test: two-session cursor visibility test
- [ ] Test: simultaneous entity move convergence test

Layer 3: Widget Runtime

WidgetFrame (iframe sandbox)

- [ ] `<WidgetFrame>` component — renders widget in sandboxed iframe with strict CSP
- [ ] `srcdoc` injection — widget HTML loaded as blob, not remote URL (security)
- [ ] Widget SDK injected into every iframe via script tag
- [ ] Lifecycle: load → init → READY → run → unmount
- [ ] Error boundary: widget crash caught, host shows error state, bus unaffected
- [ ] Size/resize: widget frame is resizable, widget receives resize events

Widget Bridge Protocol (postMessage)

- [ ] `host → widget`: emit event, resize notification, config update, theme update
- [ ] `widget → host`: emit event, setState, getState, register manifest, signal READY
- [ ] All messages typed and validated (zod schemas)
- [ ] Origin validation on all incoming messages

Widget SDK (injected into iframes)

- [ ] `StickerNest.emit(type, payload)` — emit event to host bus
- [ ] `StickerNest.subscribe(type, handler)` — subscribe to bus events
- [ ] `StickerNest.setState(key, value)` — persist widget state via host
- [ ] `StickerNest.getState(key)` — retrieve persisted state
- [ ] `StickerNest.getConfig()` — get user config for this widget instance
- [ ] `StickerNest.register(manifest)` — declare event contract on load
- [ ] `StickerNest.ready()` — signal widget is initialized

Built-in Inline Widgets (trusted, no sandbox)

- [ ] Sticky Note widget
- [ ] Clock / Timer widget
- [ ] Counter widget

- [ ] Image viewer widget
- [ ] Markdown note widget
- [ ] Each built-in uses same SDK interface as sandboxed widgets

Built-in Media & Creative Widgets

- [ ] **Audio widget** — plays audio from a File DataSource or URL. Controls: play/pause/stop/loop/volume. Emits `audio.play`, `audio.pause`, `audio.complete`. Can target an AudioEntity on canvas or play headlessly.
- [ ] **Vector widget** — creates and edits VectorEntity objects on the canvas. Path tools, fill/stroke, boolean ops, SVG import/export. Declares capability for `vector` entity type.
- [ ] **3D Object widget** — places and configures Object3DEntity objects. GLTF/GLB/OBJ import, material editor, transform controls. In 2D canvas mode renders as a preview thumbnail; full 3D in Spatial/VR mode.
- [ ] **Audio entity** — canvas renderer handles AudioEntity: icon shown at position, click to play/pause, volume control. Spatial audio in 3D/VR (position-based attenuation).
- [ ] **Vector entity renderer** — canvas renderer renders VectorEntity as SVG paths correctly at all zoom levels.
- [ ] **3D Object entity renderer** — 2D mode: renders as flat thumbnail placeholder. Spatial mode: renders as Three.js Object3D.

Layer 2: Widget Lab

- [ ] Full-screen IDE view (Monaco editor, single-file HTML widget format)
- [ ] Live preview pane — renders widget in Runtime sandbox in real time
- [ ] Preview modes: 2D isolated, 2D canvas context, 3D spatial context
- [ ] Event inspector panel — shows events emitted/received during preview
- [ ] Node graph view — visual composer for widget event logic (no-code path)

- [ ] AI generation panel — prompt → Claude generates widget HTML → loads in preview
- [ ] Manifest editor — GUI for declaring event contract and config schema
- [ ] Version history — save snapshots, restore previous versions
- [ ] Publish pipeline: validate → test → thumbnail screenshot → submit to Marketplace
- [ ] Widget import: load any widget from Marketplace into Lab for forking

---

Layer 4A: Canvas Editor (2D)

Layer 4A is split into four sub-layers. Each sub-layer is independently testable, can be built in sequence, and has a single responsibility. They share the same canvasStore and event bus but do not reach into each other's internals.

4A-1: Canvas Core (Rendering, Input, Transforms)

This is the foundation — the canvas surface itself and everything needed to see and manipulate entities on it.

Rendering & Viewport

- [ ] Infinite 2D canvas (pannable, zoomable) using HTML Canvas or Pixi.js
- [ ] Entity renderer — renders all entity types correctly
- [ ] Sticker renderer — image/GIF/video on canvas
- [ ] Canvas background (color, image, pattern, map layer)
- [ ] Snap to grid (configurable grid size)

Selection & Transforms

- [ ] Selection system — click to select, box select, multi-select
- [ ] Transform handles — move, resize, rotate per entity
- [ ] Z-order controls — bring forward, send back, lock layer

Input System

- [ ] Full keyboard shortcut map (V=select, H=hand/pan, T=text, S=shape, Esc=deselect, Cmd+Z=undo, etc.)
- [ ] Gamepad API support — thumbstick pans, face buttons map to tools
- [ ] Keyboard shortcut reference panel (? key toggles)

4A-1 Testing Gate

- [ ] Canvas renders, pans, and zooms smoothly at 60fps with 100 entities
- [ ] Select, move, resize, rotate entity with keyboard and mouse
- [ ] All keyboard shortcuts functional

4A-2: Tools & Content (Stickers, Entities, Hierarchy, Settings)

Building tools and content management that sit on top of the canvas core.

Sticker Tools

- [ ] Sticker asset browser (search, categories, upload custom)
- [ ] Drag sticker from browser onto canvas → creates Sticker entity
- [ ] Sticker click in edit mode → open config panel (set widgetId, label, size)
- [ ] Sticker click in view mode → launch associated widget
- [ ] GIF and video stickers auto-play

Entity Tools

- [ ] Text entity: click to place, double-click to edit inline
- [ ] Image entity: drag image file onto canvas
- [ ] Shape entity: draw rect, circle, polygon tools
- [ ] Audio entity: upload audio, playback controls visible on canvas

- [ ] Widget Container entity: place a running widget directly on canvas (not via sticker)

Hierarchy Panel

- [ ] Tree view of all entities on canvas
- [ ] Drag to reparent
- [ ] Visibility toggle per entity
- [ ] Lock toggle per entity
- [ ] Group/ungroup selection

Canvas Settings

- [ ] Canvas name and slug editor
- [ ] Sharing settings (private / link-only / public)
- [ ] Collaborator invite (by email, sets edit or view permission)
- [ ] Canvas thumbnail — auto-generated on save, manually overridable
- [ ] Canvas as SaaS settings — enable public access, set pricing, connect Stripe

Sticker Script Attachment

- [ ] Sticker entity gains optional `scriptId?` field alongside `widgetId?`
- [ ] Script attached to sticker fires on: click, hover-enter, hover-exit, proximity (configurable)
- [ ] Script attachment UI: in sticker config panel, "Attach script" button opens script editor or script picker
- [ ] Attached script has full SDK access: emit events, call integrations, update sticker config
- [ ] Use case: clicking a sticker emits a custom event that triggers a pipeline, without needing a full widget

4A-2 Testing Gate

- [ ] Place sticker, attach widget, click sticker → widget launches

- [ ] Create and edit all entity types
- [ ] Hierarchy panel reflects entity tree, reparent works

4A-3: Pipelines & Wiring (Pipeline Graph, AI Command Bar, Terminal, Scripts)

The interaction layer — visual wiring, CLI access to the bus, and AI-powered commands. This sub-layer shares execution infrastructure with the Event Bus Terminal and AI Command Bar (defined in their own sections below) but owns the pipeline graph UI.

Pipeline Wiring UI

- [ ] Connection graph overlay (toggle on/off)
- [ ] Pipeline is a chain of nodes: Widget → (optional Script/Filter/Transform nodes) → Widget
- [ ] Click any widget instance → see its emitted and subscribed event ports
- [ ] Drag from an emit port to a subscribe port → creates a direct Widget→Widget pipeline
- [ ] Insert intermediate node: right-click a pipeline wire → insert Script, Filter, or Transform node
- [ ] **Script node**: inline JS runs in sandboxed runtime, can transform or conditionally forward payload
- [ ] **Filter node**: GUI condition builder (if field X equals Y, pass / drop)
- [ ] **Transform node**: GUI field mapper (map source.fieldA → target.fieldB)
- [ ] Pipeline detail panel — view/edit the full node chain
- [ ] Pipeline list view — all active pipelines on this canvas
- [ ] Pipeline naming: user can label a pipeline for readability
- [ ] Pipelines are saveable as templates and reusable across canvases

4A-3 Testing Gate

- [ ] Create Widget→Widget pipeline via drag, event flows through it
- [ ] Insert Script node, transform payload, verify output
- [ ] AI Command Bar and Terminal both execute on the same bus

---

4A-4: Panels & Containers (Panel System, Docker, Workspace Chrome)

The workspace shell that surrounds the canvas — panels, dockers, and UI chrome.

Panel System

- [ ] Panel host: a UI surface anchored to the workspace edge (not on the canvas itself)
- [ ] Add panel button: opens picker with four panel types — Docker, Web Preview, Canvas Preview, Terminal
- [ ] **Web Preview panel**: renders an external URL in an iframe inside the panel
- [ ] **Canvas Preview panel**: renders another canvas live and interactively inside the panel
- [ ] **Terminal panel**: Event Bus Terminal (see Terminal section)
- [ ] Multiple panels can be open simultaneously
- [ ] Panels are resizable, collapsible, and repositionable (left, right, bottom edges)
- [ ] Panel state (open/closed, size, type, content) saved per canvas

Docker (Panel + Widget Type)

- [ ] Docker panel: system-level parent container widget, available from the panel picker
- [ ] Docker layout modes: **Tabbed** (each child gets a named tab) and **Scrollable** (children stacked vertically)
- [ ] Drag any canvas widget into the Docker → widget becomes a child of the Docker

- [ ] Drag widget out of Docker → widget returns to canvas as standalone entity
- [ ] Docker children appear in hierarchy panel as children of the Docker entity
- [ ] Children communicate via event bus as normal — Docker does not mediate events
- [ ] **Docker as widget type**: `supportsChildren: true` in manifest → any widget can be a Docker variant
- [ ] Widget Lab: Docker widget template available as starting point for custom Dockers
- [ ] Custom Docker example: media toolbar Docker, game HUD Docker, admin dashboard Docker
- [ ] **Saved Dockers**: user saves a Docker + its children as a named configuration
- [ ] Saved Dockers appear in a Docker library, openable on any canvas
- [ ] Docker nesting: a Docker can contain another Docker (max 2 levels deep)
- [ ] Desktop Mode: system Docker renders as the dock (macOS-style icon bar at bottom)
- [ ] Scene Mode: system Docker renders as a collapsible side panel tray

4A-4 Testing Gate

- [ ] Open panel, resize, reposition, close — state persists on reload
- [ ] Docker accepts child widgets, children communicate via bus
- [ ] All four panel types render correctly

Layer 4B: Spatial / VR Canvas

Three.js Renderer

- [ ] Replace 2D canvas with Three.js scene for spatial mode
- [ ] Entity hierarchy maps to Three.js Object3D hierarchy
- [ ] Stickers render as textured planes in 3D space

- [ ] Widget containers render as floating panels (CSS3DRenderer or mesh + canvas texture)
- [ ] Lighting: ambient + directional, configurable per canvas
- [ ] Skybox / environment support

WebXR / Quest 3

- [ ] WebXR session initialization (immersive-vr mode)
- [ ] Hand tracking API: pinch gesture = select/click, open hand = pan, two-hand pinch = scale
- [ ] Controller input: trigger = select, grip = grab/move, thumbstick = locomotion
- [ ] Teleportation locomotion (point and click to move)
- [ ] Comfort settings: vignette on movement, snap turning option
- [ ] Widget panels float in space, lockable to world or to hand
- [ ] VR-optimized UI: large hit targets, no hover states, gaze + pinch fallback
- [ ] Test on Quest 3 browser (Meta Quest Browser, WebXR support)

Spatial Canvas Editor

- [ ] Mode toggle: 2D ↔ Spatial ↔ VR (same canvas, different renderer)
- [ ] Place entities at depth (z-position in 3D space)
- [ ] Orbit controls in spatial preview (non-VR)
- [ ] 3D-specific entity types: 3D model (GLTF), point light, spatial audio source

---

Layer 5: Marketplace

- [ ] Widget listing page (name, description, author, version, price, screenshots)
- [ ] Search and filter (by category, widget type, price, rating)
- [ ] Install flow: free (one click) and paid (Stripe checkout)
- [ ] Installed widgets appear in `widgetStore.registry`

- [ ] Creator dashboard: revenue, installs, ratings, version management
- [ ] Rating and review system
- [ ] Collections/bundles: curated sets of widgets
- [ ] Featured and trending sections
- [ ] Platform fee: configurable % taken on paid widget sales
- [ ] Canvas as SaaS listing: canvases can be listed and sold like widgets

---

Layer 6: Platform Shell

Platform Navigation

The main navigation has five top-level destinations:

- [ ] **Canvas Gallery** — browse and manage canvases
- [ ] **Library** — unified asset, widget, pipeline, script, Docker, and template browser
- [ ] **Database Manager** — DataSources and external connections
- [ ] **Marketplace** — discover and install widgets, sticker packs, pipelines, templates
- [ ] **Settings** — account, billing, integrations, preferences

Navigation is persistent across all platform pages. Within the canvas editor, Library and Database Manager are also accessible as collapsible slide-out panels so users never have to leave the canvas to access their assets.

Canvas Gallery

- [ ] Grid view of all user canvases (thumbnail, name, slug, last edited)
- [ ] Create new canvas (blank or from template)
- [ ] Duplicate, rename, delete canvas
- [ ] Public canvas discovery (opt-in)

- [ ] Template gallery (pre-built canvases: morning dashboard, game world, project board)

Auth UI

- [ ] Sign up / sign in (email + OAuth)
- [ ] Email verification
- [ ] Password reset
- [ ] Profile page (avatar, display name, bio, public canvas list)
- [ ] Plan management (upgrade, downgrade, billing history)

Settings

- [ ] Account settings
- [ ] Canvas defaults (grid size, background, snap)
- [ ] Notification preferences
- [ ] Connected integrations (Google, GitHub, Notion, Spotify, etc.)
- [ ] API key management (for developers building against the protocol)
- [ ] Data export (download all canvases and widget data)

Database Manager

- [ ] Database Manager page in main navigation (alongside Canvas Gallery, Settings)
- [ ] **My Databases tab**: list all DataSources user owns or has access to
- [ ] Filter by type: doc, table, note, folder, file, custom
- [ ] Filter by scope: canvas, user, shared, public
- [ ] Create new DataSource directly from Database Manager (not through a widget or canvas)
- [ ] Schema editor for `table` and `custom` types: define columns, field types, required fields

- [ ] Scope and permission editor per DataSource: set ACL independently of canvas roles
- [ ] Usage view per DataSource: which canvases and widget instances are referencing it
- [ ] Inline content preview for doc, note, table types
- [ ] Rename, duplicate, delete, change scope for owned DataSources
- [ ] Share DataSource directly with another user (email invite, role assignment)
- [ ] **Connections tab**: all external integrations in one place
- [ ] Connect new service: OAuth or API key flow (Notion, Google Sheets, Airtable, Supabase, REST)
- [ ] Connection status per service: connected, syncing, error, disconnected
- [ ] Last synced time per connection
- [ ] Mapping view: which DataSources are synced from which external source
- [ ] Manual re-sync button per connection
- [ ] Re-auth flow: prompt when OAuth token expires
- [ ] Remove connection: revokes access and marks associated DataSources as disconnected
- [ ] Credentials stored encrypted server-side, never surfaced to client or widgets
- [ ] Test: DataSource created in Database Manager appears as connectable in widget config on canvas
- [ ] Test: Deleting a DataSource marks referencing widgets as disconnected without crashing them

---

Theming System

Widget Skins & Themable Tokens

- [ ] Define the widget `themeContract` field in the Widget manifest spec

- [ ] Standard token set: `--sn-bg`, `--sn-surface`, `--sn-accent`, `--sn-text`, `--sn-text-muted`, `--sn-border`, `--sn-radius`, `--sn-font-family`
- [ ] Host injects token values into widget iframe via postMessage on load and on theme change
- [ ] Widget SDK: `StickerNest.onThemeChange(handler)` callback
- [ ] Skin format: named JSON object of token → value pairs
- [ ] Per-instance skin: user can apply a skin to a single widget on the canvas
- [ ] Per-canvas skin: apply a skin to all widgets on a canvas simultaneously
- [ ] Built-in skins per widget: widget creator can ship named skins in the manifest
- [ ] Skin editor UI: token picker panel in the canvas editor sidebar
- [ ] Skins publishable and installable via Marketplace
- [ ] Skin inheritance: widget falls back to canvas theme, then to StickerNest global theme

StickerNest UI Theming

- [ ] Global design token file: all platform UI tokens defined in one place (`tokens.css`)
- [ ] Token categories: `--sn-color-*`, `--sn-radius-*`, `--sn-spacing-*`, `--sn-font-*`, `--sn-shadow-*`, `--sn-motion-*`
- [ ] Built-in themes: Dark (default), Light, High Contrast, Midnight, Sand
- [ ] Theme switcher in user settings
- [ ] Custom theme builder: user can override any token and save as a named theme
- [ ] Canvas-locked theme: creator can lock a canvas to a specific theme for branded experiences
- [ ] Enterprise white-label: full UI token override via org-level settings
- [ ] Shared token naming convention between UI theming and widget token system (so widgets inherit canvas look naturally)

- [ ] Theme tokens exposed to Widget SDK so widgets can read current theme values
- [ ] VR/Spatial theme variant: token overrides optimized for legibility in 3D/VR context

---

Security & Encryption

- [ ] All data in transit: HTTPS + WSS only
- [ ] Supabase RLS enforced on every table — no client can query data it doesn't own
- [ ] Widget sandbox CSP: no access to parent DOM, no external network by default
- [ ] Widget postMessage origin validation on every message
- [ ] Canvas sharing tokens: link-sharing uses signed, expiring tokens (not just URL)
- [ ] Private canvases: content never returned to non-authorized users, even via API
- [ ] Encrypted widget state: sensitive widget configs (API keys user enters) encrypted at rest
- [ ] Auth tokens: short-lived JWTs + refresh token rotation
- [ ] Rate limiting on all API routes
- [ ] Content moderation pipeline for Marketplace submissions
- [ ] GDPR: data deletion flow, data export, cookie consent

StickerNest as Protocol / OS

Protocol Specification (publish separately)

- [ ] Define the Widget Manifest schema (open spec, versioned)

- [ ] Define the Event Bus message format (open spec)
- [ ] Define the Widget Bridge postMessage protocol (open spec)
- [ ] Define the Canvas state schema (open spec)
- [ ] Publish spec docs at `protocol.stickernest.app`

Third-Party Runtime Support

- [ ] Any developer can implement the protocol and build widgets that work in StickerNest
- [ ] Widget validation tool: checks a widget against the protocol spec
- [ ] Official SDK package: `@stickernest/sdk` (npm) — the Widget SDK as an npm package
- [ ] Official types package: `@stickernest/types` (npm) — shared TypeScript types

OS-Layer Concepts

- [ ] Widget permissions model: widgets declare what they need (network, storage, entity access) in manifest
- [ ] User grants/denies permissions at install time
- [ ] Widget inter-communication is always mediated by the bus — no direct access
- [ ] Canvas = environment. Widget = app. Event bus = IPC. Marketplace = app store.

Web, Mobile & Touch

Web and mobile are both launch requirements, not nice-to-haves. The canvas editor, canvas gallery, and view-mode canvases must work excellently on desktop browsers, phones, and tablets at launch. Web (desktop browser) is the primary authoring surface; mobile is a co-equal consumption and light-editing surface.

Touch Input System

- [ ] Tap = select / activate (replaces click)
- [ ] Tap + hold = context menu (replaces right-click)
- [ ] Single-finger drag = move selected entity
- [ ] Two-finger drag = pan canvas
- [ ] Two-finger pinch = zoom canvas
- [ ] Two-finger rotate = rotate selected entity
- [ ] Double-tap = edit inline (text entities)
- [ ] Touch event system abstracted behind a unified pointer layer (same code handles mouse, touch, pen)

Mobile Layout

- [ ] Bottom sheet replaces sidebars and panels (slides up from bottom)
- [ ] Widget docker collapses to a floating action button (FAB)
- [ ] Toolbar moves to bottom of screen (thumb-reachable zone)
- [ ] Hierarchy panel accessible via swipe-up drawer
- [ ] Canvas settings accessible via bottom sheet
- [ ] Responsive breakpoints: phone (< 640px), tablet (640–1024px), desktop (> 1024px)
- [ ] All tap targets minimum 44×44px

Mobile-Specific Considerations

- [ ] Widget Lab is desktop browser-first (redirect to desktop prompt on mobile)
- [ ] Sticker placement via tap-to-place flow (no drag from sidebar on small screens)
- [ ] Pipeline wiring UI simplified on mobile (list-based, not graph-based)
- [ ] VR mode not available on mobile (shows "Open on Quest 3" prompt)
- [ ] Camera/photo library access for uploading sticker assets on mobile

- [ ] PWA manifest: installable as home screen app on iOS and Android
- [ ] iOS Safari: test WebKit quirks (overscroll, viewport height, safe areas)
- [ ] Android Chrome: test touch event handling, keyboard viewport resize behavior

Mobile Testing Gate

- [ ] Canvas gallery loads and is usable on iPhone SE (smallest target)
- [ ] Sticker placed and moved on iPad with touch
- [ ] Widget launches from sticker tap on mobile
- [ ] Canvas saves and reloads correctly on mobile
- [ ] Two-finger zoom and pan work without conflict

Web (Desktop Browser)

Web is the primary authoring environment. All core creation flows — canvas editing, Widget Lab, pipeline wiring, Database Manager — must be optimized for mouse + keyboard on desktop browsers first.

- [ ] Full canvas editor experience with mouse, keyboard shortcuts, and hover states
- [ ] Widget Lab is a desktop-first experience (Monaco editor, multi-panel layout)
- [ ] Pipeline wiring uses the full graph-based UI on desktop
- [ ] All panel types (Docker, Web Preview, Canvas Preview, Terminal) available and resizable
- [ ] Multi-panel layouts: sidebars, bottom panels, floating panels — all fully functional
- [ ] Drag-and-drop from Library onto canvas works with mouse
- [ ] Right-click context menus available throughout
- [ ] Keyboard shortcut map fully operational (V, H, T, S, Cmd+Z, `, /, etc.)
- [ ] Browser targets: Chrome (primary), Firefox, Safari, Edge — latest two versions each

- [ ] Responsive layout adapts between narrow desktop windows and ultrawide monitors
- [ ] Copy/paste support: paste images, text, URLs onto canvas
- [ ] Browser tab title reflects active canvas name
- [ ] Deep linking: all routes (canvas, library, settings) are bookmarkable and shareable URLs

Web Testing Gate

- [ ] Full canvas editing flow works in Chrome, Firefox, Safari, and Edge
- [ ] Widget Lab: create, edit, preview, and publish a widget entirely in browser
- [ ] Pipeline wiring: create a multi-node pipeline using the graph UI
- [ ] All keyboard shortcuts function correctly
- [ ] Canvas loads under 3s on a standard broadband connection
- [ ] No layout breakage between 1024px and 2560px viewport widths

---

Interaction Stack Clarification

> These four systems share the same underlying infrastructure. Agents must not build them as separate systems — they are four interfaces to the same event bus.

> **Pipelines** → no-code visual wiring (Layer 4A-3)

> **AI Command Bar** → natural language frontend to the terminal (Layer 4A-3)

> **Event Bus Terminal** → direct CLI to the bus (Layer 4A-3, developer tool)

> **Scripts** → persistent logic on the bus (Layer 3, headless widgets)

AI Command Bar

- [ ] Command bar component: floating input, triggered by `/` key or tap-and-hold on empty canvas
- [ ] Intent parser: Claude interprets natural language → generates terminal commands → executes them
- [ ] Command bar shares execution infrastructure with Event Bus Terminal (not a separate emitter)
- [ ] Action registry: all bus events and canvas actions registered with natural language descriptions
- [ ] Command bar can target all widgets, a specific widget, or a filtered set
- [ ] Voice input: microphone button (Web Speech API)
- [ ] Command history: up arrow cycles through recent commands
- [ ] Suggestions: matching actions shown as user types
- [ ] Script generation: "create a script that..." → Claude generates script → opens in terminal for review before saving
- [ ] Mobile: triggered by FAB long-press
- [ ] Available in both view mode (widget control) and edit mode (canvas editing)

Event Bus Terminal

- [ ] Terminal panel type: added via panel picker alongside Docker, Web Preview, Canvas Preview
- [ ] Keyboard shortcut: backtick (`` ` ``) toggles terminal overlay (consistent with game engine convention)
- [ ] Command parser: tokenize and validate terminal commands before execution
- [ ] Core commands: `emit`, `subscribe`, `list widgets`, `list scripts`, `list pipelines`, `run script`, `history`, `clear`
- [ ] `emit <event.type> <JSON payload>` — emits directly to the bus
- [ ] `subscribe <pattern>` — opens a live event stream in the terminal output (Ctrl+C to stop)

- [ ] `history <n>` — shows last N events from the bus ring buffer
- [ ] `list widgets` — shows all registered instances with their event contracts
- [ ] `run script <name>` — manually executes a saved script
- [ ] Terminal output is color-coded: events in blue, errors in red, system messages in gray
- [ ] One-liner script authoring: type a script inline, `save as <name>` to persist it
- [ ] Terminal is scoped to current canvas — cannot affect other canvases or system internals
- [ ] Permission enforcement: terminal respects canvas role — Viewers cannot emit or mutate
- [ ] Security hard limits: no file system access, no shell commands, no DOM access outside sandbox, no bypassing RLS
- [ ] Terminal history persisted per canvas session (not across sessions)
- [ ] AI Command Bar uses terminal execution layer internally — both share the same command executor
- [ ] Test: emit event from terminal → verify widget receives it
- [ ] Test: Viewer-role user cannot emit mutations via terminal

---

CI/CD & Branch Strategy

> This must be set up before any agent writes implementation code. All agents depend on it.

Deployment

- [ ] Frontend: Vercel. `main` branch auto-deploys to production. `develop` branch auto-deploys to staging.
- [ ] Backend: Supabase (managed). Schema changes via migrations only — never direct table edits in production.
- [ ] CDN: Cloudflare in front of Supabase Storage for all public buckets.

☐ Environment variables: `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_KEY`, `STRIPE_SECRET_KEY`, `ANTHROPIC_API_KEY` — all set in Vercel environment settings, never in code.

Branch Strategy

☐ `main` — production. Protected. No direct pushes. Merge from `develop` only after staging sign-off.

☐ `develop` — integration branch. All agent feature branches merge here. Auto-deploys to staging.

☐ `feat/kernel` — Kernel agent branch

☐ `feat/runtime` — Runtime agent branch

☐ `feat/canvas-core` — Canvas Core agent branch (4A-1: rendering, input, transforms)

☐ `feat/canvas-tools` — Canvas Tools agent branch (4A-2: stickers, entities, hierarchy, settings)

☐ `feat/canvas-wiring` — Canvas Wiring agent branch (4A-3: pipelines, AI bar, terminal)

☐ `feat/canvas-panels` — Canvas Panels agent branch (4A-4: panel system, Docker, chrome)

☐ `feat/spatial` — Spatial agent branch

☐ `feat/social` — Social agent branch

☐ `feat/lab` — Widget Lab agent branch

☐ `feat/shell` — Shell agent branch

☐ `feat/market` — Marketplace agent branch

Agent Unblocking Order

Agents cannot start until their dependencies are ready. The order:

1. **Kernel agent starts first** — produces `@sn/types` package and event bus. All other agents are blocked until this is done.

2. **Runtime + Social agents start in parallel** — both depend on Kernel types only.
3. **Canvas Core (4A-1) + Spatial agents start in parallel** — depend on Kernel + Runtime. 4A-1 must complete before other 4A sub-layers begin.
4. **Canvas Tools (4A-2), Canvas Wiring (4A-3), Canvas Panels (4A-4) start in parallel** — all depend on Kernel + Runtime + 4A-1 (Canvas Core). These three sub-layers do not depend on each other.
5. **Lab + Shell + Market agents start in parallel** — depend on Kernel + Runtime + 4A-1. Lab also benefits from 4A-3 (pipeline wiring UI) but can stub it.

CI Pipeline (runs on every PR to `develop`)

- [ ] TypeScript type check (zero errors required)
- [ ] Unit tests (event bus, store logic, DataSource CRUD)
- [ ] Integration tests (Supabase schema, auth flow)
- [ ] Performance budget check (bundle size, Lighthouse CI)
- [ ] a11y audit (axe-core automated scan)
- [ ] No PR merges with failing CI

Types Package

- [ ] Kernel agent publishes `@sn/types` as an internal npm package (or path alias) before any other agent starts
- [ ] All agents import from `@sn/types` — never redefine shared types locally
- [ ] Any change to `@sn/types` requires a PR reviewed by all affected agents before merge

---

Resolved Architecture Decisions

> These decisions are final. Agents must not re-litigate them.

Desktop Mode Entity Transition

When a canvas switches to Desktop Mode, every existing widget container automatically becomes a window. Entity's current canvas size = initial window size. Entity's current canvas position = initial window position. No data lost. No user prompt. Switching back to Scene Mode restores the spatial layout.

Canvas Embedding

- [ ] Embed URL: `stickernest.app/u/{username}/{slug}?embed=true`
- [ ] Embed code generator in canvas share settings panel
- [ ] Embed is view-mode only — no editor chrome rendered when `?embed=true` is present
- [ ] Private canvases cannot be embedded (server enforced)
- [ ] Domain allowlist: canvas owner can restrict which domains may embed their canvas
- [ ] Widgets run normally inside embeds — all interactivity preserved
- [ ] Canvas as SaaS delivery: embed is the primary mechanism
- [ ] Test: embed URL renders canvas without editor chrome
- [ ] Test: private canvas embed URL returns 403

Guest / Anonymous Access

- [ ] Public canvases accessible without account in view mode
- [ ] Guest can: view canvas, interact with widgets, see other presence cursors
- [ ] Guest cannot: edit, annotate, chat, save widget state across sessions
- [ ] Guest presence label: "Guest" + random color
- [ ] Any content creation action (annotation, chat) prompts account creation
- [ ] Test: unauthenticated user loads public canvas and can interact with a widget
- [ ] Test: unauthenticated user cannot mutate any entity

URL Routing

- [ ] Route: `/u/{username}/{slug}` — standard user canvas
- [ ] Route: `/c/{slug}` — global unique slug (Creator+)
- [ ] Route: `/u/{username}/{slug}?embed=true` — embedded canvas
- [ ] Route: `/gallery`, `/library`, `/databases`, `/marketplace`, `/settings`, `/lab`
- [ ] Slug rules: alphanumeric + hyphens, max 64 chars, unique per user for standard, unique globally for `/c/`
- [ ] Slug change: old slug 301 redirects to new slug for 90 days
- [ ] Custom domain: Enterprise only, configured via DNS CNAME to Vercel

Widget Hot-Reload Policy

- [ ] Free widgets: silent update on next canvas open. Never mid-session.
- [ ] Paid / installed widgets: update-available badge on widget instance. User chooses when to apply.
- [ ] Pinned widgets (Creator+): never auto-update. User must manually unpin.
- [ ] Major version bump (breaking change): always treated as prompted update regardless of tier.
- [ ] Widget version stored in `widget_instances.version` — compared against latest on canvas load
- [ ] Update badge component: appears on widget container in canvas editor, dismissable
- [ ] Test: free widget update applies on next canvas open without prompt
- [ ] Test: paid widget shows update badge, does not auto-update

VR Spatial Events

- [ ] `BusEvent` base type gains optional `spatial?: SpatialContext` field
- [ ] `SpatialContext` = `{ position: Vector3, rotation: Quaternion, normal: Vector3 }`
- [ ] Non-VR events: `spatial` is always `undefined`

- [ ] VR interaction events: `spatial` is populated with 3D world coordinates of interaction origin
- [ ] Backward compatible: widgets that ignore `spatial` continue to work unchanged
- [ ] Spatial-aware widgets: read `spatial` to respond to 3D position (e.g. proximity triggers, directional audio)
- [ ] Test: 2D widget receives VR-emitted event and processes it correctly (ignoring spatial field)
- [ ] Test: spatial-aware widget receives correct position data from a VR pinch interaction

---

Onboarding

- [ ] First-run experience: new user lands on an interactive intro canvas (not a blank screen)
- [ ] Guided tour: step-by-step overlay highlighting canvas, sticker browser, widget docker
- [ ] Empty state: canvas gallery shows "Create your first canvas" with template suggestions
- [ ] Template gallery at canvas creation: morning dashboard, game world, project board, blank
- [ ] Tooltips on first use of each major tool (dismissable, not shown again)
- [ ] Progress indicator: "You've placed your first sticker!" type encouragements
- [ ] Onboarding checklist widget (optional, dismissable) that guides new users through key actions
- [ ] Skip option at every step — power users should never be blocked

---

Notifications

- [ ] In-app notification bell with unread count

- [ ] Notification types: collaborator joined, annotation added, widget update available, marketplace sale, canvas version saved, session invite
- [ ] Email notifications: daily digest or per-event (user configurable)
- [ ] Notification preferences panel in settings (per category, per channel)
- [ ] Realtime in-app toast for live events (collaborator joined, etc.)
- [ ] Push notifications for PWA installs (opt-in)
- [ ] Notification read/unread state persisted in Supabase

Annotations & Comments

- [ ] Annotation pin: click/tap empty canvas area to drop a comment pin
- [ ] Annotation can be attached to a specific entity or to a canvas position
- [ ] Annotation thread: multiple replies per annotation
- [ ] Resolve annotation: hides it (preserves in history)
- [ ] Annotation visibility toggle: show/hide all annotations on canvas
- [ ] Commenter role can add annotations but not edit entities
- [ ] Annotations stored in Supabase, real-time updates via social layer
- [ ] Annotation count badge shown on canvas thumbnail in gallery
- [ ] Email notification when someone annotates your canvas

Canvas Version History

- [ ] Auto-snapshot every 5 minutes while canvas is open (configurable)
- [ ] Snapshot on significant actions: first entity placed, publish, share link created
- [ ] Named versions: user can manually save a named snapshot ("v1 launch", "before redesign")
- [ ] Version browser UI: timeline sidebar, preview on hover
- [ ] Restore version: replaces current canvas state (with confirmation prompt)

- [ ] Version diff view: highlight what changed between two versions (entity added/removed/moved)
- [ ] Retention policy: last 10 auto-snapshots free, unlimited named versions on Creator+
- [ ] Version history stored as canvas state diffs in Supabase (not full copies)

Search

- [ ] In-canvas search: find entity by name/label, jump to it on canvas
- [ ] Cross-canvas search: search across all your canvases for entities, widgets, stickers
- [ ] Marketplace search: covered in Layer 5
- [ ] Search result highlights matching entities with a visual indicator
- [ ] Keyboard shortcut: Cmd+F opens in-canvas search
- [ ] Search indexes: entity names, sticker labels, widget names, annotation text

---

Import / Export

- [ ] Export canvas as PNG (full canvas or viewport only)
- [ ] Export canvas as PDF (for sharing as a static document)
- [ ] Export canvas data as JSON (full state backup, importable)
- [ ] Import canvas from JSON (restore backup or share canvas structure)
- [ ] Export widget as standalone HTML file (from Widget Lab)
- [ ] Import sticker assets in bulk (zip upload)
- [ ] Data export: download all user data (GDPR compliance) — canvases, widgets, settings

The Library

The Library is a first-class platform page in the main navigation. It is a unified view over all reusable assets and configurations the user owns — not a new storage system, just a browsing and organization layer over what already exists.

Media Section

- [ ] List all File DataSources owned by the user (images, GIFs, video, audio, SVG, 3D models)
- [ ] Filter by asset type (tabs or sidebar filter)
- [ ] Search by filename or tag
- [ ] Grid view (thumbnail) and list view (name, type, size, date uploaded)
- [ ] Upload new asset directly from Library (goes to correct bucket + creates File DataSource)
- [ ] Drag asset from Library onto open canvas — creates the appropriate entity type automatically
- [ ] Asset detail panel: preview, file info, which canvases reference it, rename, delete, share
- [ ] Bulk upload via zip file
- [ ] Assets uploaded to any canvas automatically appear in Library — no manual saving required
- [ ] Storage usage indicator: current usage vs plan limit

Stickers Section

- [ ] List all sticker collections: user-created, downloaded from Marketplace, system defaults
- [ ] Browse collection contents as a grid
- [ ] Create new collection (group of sticker assets)
- [ ] Add/remove assets from a collection
- [ ] Drag sticker from Library directly onto canvas
- [ ] Install sticker pack from Marketplace (appears here immediately)

☐ Publish sticker pack to Marketplace (from Library, not just from Marketplace UI)

Widgets Section

☐ **Installed** — all widgets from Marketplace the user has installed

☐ **Mine** — all widgets the user has authored in Widget Lab (published and draft)

☐ **Built-ins** — system-provided widgets (clock, sticky note, etc.)

☐ Each widget card shows: name, icon, version, author, install count (if published), last updated

☐ Launch widget directly from Library onto active canvas

☐ Open widget in Widget Lab (for owned/draft widgets)

☐ View widget on Marketplace page (for installed widgets)

☐ Uninstall widget (removes from `user_installed_widgets`, removes from all canvases with confirmation)

☐ Update available badge when a newer version exists

☐ Pinned version indicator for canvases using a specific version

Pipelines Section

☐ List all saved pipeline templates owned by the user

☐ Pipeline card shows: name, node chain summary (Widget A → Script → Widget B), date saved

☐ Apply pipeline to active canvas: picks matching widget instances or prompts to resolve

☐ Edit pipeline template (opens connection graph)

☐ Delete pipeline template

☐ Share pipeline template with another user

☐ Publish pipeline template to Marketplace (bundleable with widgets)

Scripts Section

- [ ] List all saved scripts owned by the user
- [ ] Script card shows: name, brief description, event contract summary, date created
- [ ] Preview script code inline
- [ ] Attach script to active canvas or specific widget from Library
- [ ] Edit script (opens script editor)
- [ ] Tag scripts for organization
- [ ] Search scripts by name or tag

Dockers Section

- [ ] List all saved Docker configurations
- [ ] Docker card shows: name, child widget list, thumbnail, date saved
- [ ] Open Docker on active canvas (restores full configuration with children)
- [ ] Edit Docker configuration
- [ ] Delete saved Docker

Templates Section

- [ ] List all canvas templates the user owns (saved from their canvases or purchased)
- [ ] Template card shows: thumbnail, name, description, entity/widget count
- [ ] Create new canvas from template (one click)
- [ ] Save current canvas as a template
- [ ] Publish template to Marketplace
- [ ] Delete template

Library General

- [ ] Global Library search: searches across all sections simultaneously
- [ ] Recent items: most recently used assets, widgets, scripts across all sections

- [ ] Favorites: star any item to pin it to a Favorites section
- [ ] Sort options: date added, name, most used, file size
- [ ] Library accessible as a slide-out panel within the canvas editor (not just as a full page)
- [ ] Library state (last active section, sort, filters) persisted per user session
- [ ] Test: asset uploaded on Canvas A appears in Library and is draggable onto Canvas B

Collaboration Roles & Permissions

- [ ] Role types: Owner, Editor, Commenter, Viewer
- [ ] Per-canvas per-user role assignment
- [ ] Invite by email (sends invite link)
- [ ] Invite via shareable link (set role for anyone with link)
- [ ] Role enforcement on all canvas mutations (server-side via RLS)
- [ ] Owner can transfer ownership to another user
- [ ] Remove collaborator from canvas
- [ ] Collaborator list view in canvas settings
- [ ] Guest access: Viewer role without requiring account (public canvases only)

Subscription Tiers & Billing

Plan Definitions

- [ ] **Free**: 3 canvases, 5 widgets installed, no realtime collaboration, no slug, no Marketplace publishing, 100MB storage
- [ ] **Pro ($12/mo)**: unlimited canvases, full widget library, realtime collab up to 5 seats, canvas slugs, version history (10 snapshots), 5GB storage

- [ ] **Creator ($29/mo)**: everything in Pro + Widget Lab, Marketplace publishing, revenue sharing, unlimited version history, 20GB storage, analytics
- [ ] **Enterprise (custom)**: everything in Creator + SSO, white-label, SLA, unlimited seats, dedicated support, custom integrations

Billing Implementation

- [ ] Stripe integration: subscription creation, upgrade, downgrade, cancellation
- [ ] Billing portal: Stripe customer portal embedded in settings
- [ ] Usage metering: track canvas count, storage, collab seats against plan limits
- [ ] Upgrade prompt: shown contextually when user hits a limit
- [ ] Free trial: 14-day Pro trial on signup (no credit card required)
- [ ] Invoices: downloadable PDF invoices in billing history
- [ ] Canvas as SaaS: Stripe Connect for creator payouts, platform fee deducted automatically

---

Offline & Local-First

- [ ] Canvas state persisted to IndexedDB on every change (not just on save)
- [ ] App loads from IndexedDB cache if network unavailable
- [ ] Offline change queue: mutations queued locally, replayed on reconnect
- [ ] Reconnect handling: merge queue with server state, surface conflicts to user
- [ ] Offline indicator: subtle UI badge when working offline
- [ ] Realtime collaboration gracefully disabled when offline (cursors hidden, edits local only)
- [ ] Service worker: cache app shell and static assets for offline load
- [ ] PWA offline mode: full canvas editing works offline for single-user canvases

---

Widget Versioning

- [ ] Widgets use semver (major.minor.patch) declared in manifest
- [ ] Canvas stores the widget version it was configured with
- [ ] Auto-update mode: canvas always runs latest compatible version (default)
- [ ] Pinned mode: canvas locked to specific version (Creator+ feature)
- [ ] Breaking change detection: Widget Lab warns creator when manifest changes break the event contract
- [ ] Deprecation notices: old widget versions show deprecation warning after 90 days
- [ ] Rollback: canvas owner can revert to a previously-used widget version
- [ ] Version history visible in Marketplace widget listing page

---

Accessibility (a11y)

- [ ] WCAG 2.1 AA compliance target for all platform UI
- [ ] Full keyboard navigation of canvas editor (select, move, resize, configure without pointer)
- [ ] Focus trap prevention: modals and panels trap focus correctly, widgets do not
- [ ] Screen reader: all interactive elements have accessible labels
- [ ] Canvas entity screen reader announcements ("Text entity selected: Hello World")
- [ ] High Contrast theme ships as a built-in theme (meets 7:1 contrast ratio)
- [ ] `prefers-reduced-motion`: all animations/transitions disabled when set
- [ ] `prefers-color-scheme`: Dark/Light theme auto-selected on first load
- [ ] Touch targets: minimum 44×44px across all UI
- [ ] Widget SDK a11y guidance: documentation + validator for widget creators
- [ ] Marketplace a11y review: basic a11y check required before widget approved

---

Internationalization (i18n)

- [ ] All UI strings externalized to translation JSON files (no hardcoded copy in components)
- [ ] i18n framework: `react-i18next` or equivalent
- [ ] RTL layout support via CSS logical properties throughout (not `left`/`right`, use `inline-start`/`inline-end`)
- [ ] Date, number, currency formatting via `Intl` APIs
- [ ] Widget manifest supports localized `name` and `description` fields
- [ ] Launch language: English only (but architecture ready)
- [ ] Community translation workflow: post-launch, translation files open-sourced for contributions

---

Cross-Canvas Widget Communication

- [ ] Define cross-canvas channel spec in Widget manifest (`crossCanvasChannels` field)
- [ ] Canvas linking UI: owner explicitly links two canvases and maps channels between them
- [ ] Permission model: both canvas owners must approve a cross-canvas link
- [ ] Single-user cross-canvas: route events via Supabase Realtime between browser tabs
- [ ] Multiuser cross-canvas: same Realtime broker, scoped to linked canvas pair
- [ ] SDK: `StickerNest.emitCrossCanvas(channel, payload)` and `StickerNest.subscribeCrossCanvas(channel, handler)`
- [ ] Latency target: cross-canvas events delivered within 500ms
- [ ] Canvas link manager UI: view and remove active cross-canvas links per canvas
- [ ] Cross-canvas event inspector: show events crossing canvas boundaries in debug mode

- [ ] Add `crossCanvas.*` event namespace to bus — distinguishable from local events
- [ ] Test: two tabs with linked canvases exchange events correctly
- [ ] Test: two users on linked canvases exchange events with correct permissions

---

Widget State Persistence

> **Clarification with DataSources:** Widget *state* (runtime instance state like a timer's current countdown) is separate from widget *data* (a doc's content, a table's rows). Data lives in a DataSource. State lives in the widget instance. Both persist across sessions but through different mechanisms.

- [ ] Instance state: `StickerNest.setState(key, value)` persists to Supabase per widget-instance-per-canvas
- [ ] Instance state: `StickerNest.getState(key)` retrieves on widget load — widget resumes exactly where it left off
- [ ] User state: `StickerNest.setUserState(key, value)` persists to Supabase per user (follows user across all canvases)
- [ ] User state: `StickerNest.getUserState(key)` retrieves on any canvas where that widget is installed
- [ ] State namespacing: instance state keys are scoped to `{widgetId}:{instanceId}` to prevent collisions
- [ ] State size limits: 1MB per widget instance, 10MB total user state (enforced server-side)
- [ ] State migration: widget manifest can declare a `stateMigration` function for handling version upgrades
- [ ] Offline: state writes queued locally (IndexedDB) and synced on reconnect
- [ ] State cleared on: widget uninstall (with confirmation), canvas delete, user account delete
- [ ] Add `state` column to `widget_instances` Supabase table (JSONB)
- [ ] Add `user_widget_state` Supabase table (userId, widgetId, key, value)

Database Widget Integration

> **With DataSources:** External databases (Notion, Google Sheets, Airtable) sync into StickerNest as DataSources of type `table` or `custom`. The widget always reads from the local DataSource — the integration layer handles keeping it in sync with the external source. Widgets never call external APIs directly.

☐ Integration registry: user connects external services in Settings (OAuth or API key per service)

☐ Supported integrations v1: Notion databases, Google Sheets, Airtable, Supabase tables, REST/GraphQL endpoints

☐ Widget manifest: `integrations` field declares which integrations the widget needs

☐ Permission prompt: user grants/denies a widget's access to a specific integration on install

☐ SDK: `StickerNest.integration(name).query(params)` — sandboxed data access, host proxies the request

☐ SDK: `StickerNest.integration(name).mutate(params)` — write back to external source

☐ Integration proxy: all external calls go through the StickerNest backend (widget never holds credentials)

☐ Integration credentials stored encrypted in Supabase, never exposed to widget iframe

☐ Built-in database widgets: Notion table view, Google Sheets grid, Supabase table viewer

☐ Generic REST widget: user provides endpoint + auth header, widget renders JSON response

☐ Integration connection status: widget shows connected/disconnected state

☐ Re-auth flow: graceful prompt when OAuth token expires

☐ Webhook support: widgets can register webhooks so external services push updates to the event bus

---

Widget Scripting

☐ Script runtime: lightweight sandboxed JS environment (same iframe security model as widgets)

☐ Script SDK: full Widget SDK available in scripts (emit, subscribe, setState, integration access)

☐ Script editor: lightweight mode in Widget Lab (no full IDE needed — just a code textarea + event inspector)

☐ AI script generation: command bar supports "create a script that..." → Claude generates it

☐ Script manifest: minimal version of widget manifest (name, eventContract, no UI)

☐ Scripts run headlessly — no visible UI, no canvas presence

☐ Scripts appear in the pipeline wiring UI as connectable nodes

☐ Scripts can be attached to a canvas (run whenever canvas is open) or to a specific widget instance

☐ Script library: save and reuse scripts across canvases

☐ Common script templates: event logger, payload transformer, conditional forwarder, scheduler

☐ Script error handling: errors caught and surfaced in event inspector, script halted (does not crash bus)

☐ Scripts not publishable to Marketplace standalone — bundleable with widgets or canvas templates

---

Desktop Mode & Canvas View Modes

Canvas View Mode System

☐ View mode stored per canvas in `canvases.viewMode` (scene | desktop)

☐ Mode switcher UI: toggle in canvas toolbar (or canvas settings)

- [ ] Mode preference saved and restored on canvas open
- [ ] Both modes share identical underlying data: same entity hierarchy, same bus, same widget runtime
- [ ] Mode switch is non-destructive: switching back and forth never loses data
- [ ] Future mode stubs: `board` (kanban columns), `grid` (snapped spreadsheet layout)

Scene Mode (default)

- [ ] Existing canvas editor behavior (see Layer 4A)
- [ ] Stickers and entities placed freely on infinite canvas
- [ ] No window chrome, no taskbar — pure spatial composition

Desktop Mode

- [ ] Canvas background renders as desktop wallpaper (solid color, image, or gradient — set in canvas settings)
- [ ] Widget containers render as OS-style windows: title bar, minimize, maximize, close buttons
- [ ] Windows are resizable and draggable within the canvas viewport
- [ ] Window snapping: drag to screen edge to snap to half or quarter of viewport
- [ ] Window stacking: z-order managed by last-focused window (click to bring to front)
- [ ] Minimize: window collapses to taskbar entry
- [ ] Maximize: window fills viewport (with restore button)
- [ ] Taskbar: horizontal bar (top or bottom, user preference) showing open/minimized windows
- [ ] Dock: widget docker renders as a macOS-style dock with icon + label per pinned widget
- [ ] Stickers in Desktop Mode render as desktop icons (image + label below)

- [ ] Double-click sticker icon → launches associated widget as a new window
- [ ] Desktop right-click context menu: new widget, new folder, new doc, arrange icons, change wallpaper
- [ ] Icon grid snap: sticker icons snap to invisible grid on desktop
- [ ] Multi-monitor stub: future support for multiple viewport "monitors" on one canvas

Window State Persistence

- [ ] Window position stored in `WidgetContainerEntity.transform` (shared, persisted to canvas like all entity transforms)
- [ ] Window UI state stored in `WidgetContainerEntity.windowState` (minimized, maximized, width, height) — see TypeScript Interface Contracts
- [ ] `windowState` is `undefined` in Scene Mode; populated only in Desktop Mode
- [ ] Window chrome preferences (taskbar position, snap behavior) are per-user UI state, stored in `uiStore`

Parent/Child Widget Nesting

- [ ] Widget manifest: `supportsChildren: true` flag enables a widget to host child widgets
- [ ] Child slot system: parent declares named slots (e.g. `main`, `sidebar`, `tab-1`) that accept child widgets
- [ ] Canvas editor: drag a widget into a parent widget container to nest it
- [ ] Each child widget has its own SDK context, state, and event bus access (fully independent)
- [ ] Parent controls child frame layout (size, position within parent) — not child content
- [ ] Events flow normally through the bus — children do not communicate through parent, only through bus

- [ ] Drag out: child can be dragged out of parent to become a standalone canvas entity
- [ ] Drag in: standalone widget can be dragged into a compatible parent
- [ ] Nesting depth: max 3 levels deep (enforced to prevent performance issues)
- [ ] Entity hierarchy reflection: nested widgets appear as children in the hierarchy panel
- [ ] Built-in parent widgets: Dashboard (grid layout), Tab Container (tabbed layout), Splitview (two-pane), Folder

Doc Widget

- [ ] Rich text editor (ProseMirror or TipTap) with: headings, paragraphs, lists, checkboxes, code blocks, inline images, embeds
- [ ] Autosave: content saved to widget instance state on every change (debounced 500ms)
- [ ] Collaborative editing: if canvas is multiuser, doc widget supports real-time co-editing via Yjs CRDT
- [ ] Export: export doc as Markdown, PDF, or plain text
- [ ] Link to external source: doc backed by a Notion page or Google Doc via integration layer
- [ ] Doc widget emits events: `doc.updated`, `doc.heading-changed`, `doc.word-count` (useful for pipelines)
- [ ] Embeds: paste a URL to embed another widget, image, or video inline in the doc

Folder Widget

- [ ] Folder renders as a sticker icon in Scene Mode (click to open as panel)
- [ ] Folder renders as a desktop icon in Desktop Mode (double-click to open as window)
- [ ] Contents: any widget type, nested folders, sticker assets

- [ ] Views: grid view (icon grid) and list view (name + type + date)
- [ ] Drag to add: drag any widget or sticker into the folder to add it
- [ ] Drag out: drag item out to remove from folder (does not delete, returns to canvas)
- [ ] Folder path: breadcrumb navigation for nested folders
- [ ] Folder contents are stored in a **Folder DataSource** (data layer). The Folder widget is a view over that DataSource. Widget instance state stores only view/UI preferences (sort order, view mode, scroll position).
- [ ] FolderItem kinds: DataSource reference, widget shortcut, canvas link, external link
- [ ] Drag into folder default behavior: **MOVE** (creates FolderItem and removes original desktop sticker). Later: modifier-key can create shortcut while keeping original sticker.
- [ ] Folder widget emits events: `folder.opened`, `folder.item-added`, `folder.item-removed`

Performance Budgets (CI-enforced)

- [ ] Canvas FPS: 60fps for up to 100 entities on mid-range hardware; 30fps minimum floor
- [ ] Time to interactive: under 3 seconds on 4G connection
- [ ] Widget load: iframe sandbox initialized and READY signal within 500ms
- [ ] Event bus latency: emit-to-handler under 1ms for local events
- [ ] Realtime cursor sync: visible to remote user within 200ms
- [ ] Main bundle: under 200kb gzipped; per-layer code splitting enforced
- [ ] Mobile parity: all above targets must hold on a mid-range Android device
- [ ] Lighthouse scores: Performance 90+, Accessibility 90+, PWA 100

CDN & Asset Delivery

- [ ] Sticker assets (images, GIFs, videos) served from CDN (Supabase Storage + Cloudflare)
- [ ] Widget bundles cached at CDN edge, invalidated on new version publish
- [ ] Canvas thumbnails served from CDN
- [ ] Image optimization: auto-convert uploaded sticker images to WebP
- [ ] GIF optimization: compress on upload, serve as WebM video where supported
- [ ] Lazy loading: sticker assets loaded only when in or near viewport
- [ ] Asset size limits: sticker images 5MB max, GIFs 15MB max, videos 50MB max

Analytics

- [ ] Platform analytics (internal): DAU, canvas creation rate, widget installs, realtime session counts
- [ ] Creator analytics (user-facing): canvas views, widget installs, revenue, top events emitted
- [ ] Canvas analytics: view count, unique visitors, average session duration (for public/SaaS canvases)
- [ ] Error tracking: Sentry integration for client-side errors and widget crashes
- [ ] Performance monitoring: real-user metrics (Core Web Vitals) tracked in production
- [ ] Privacy-first: no third-party tracking pixels; all analytics first-party or self-hosted

---

Build Order & Milestones

Milestone 0 — Foundation (Week 1-2)

Kernel agent completes Layer 0. All types defined. Event bus tested. Stores implemented. Supabase schema live with RLS. Auth working. Performance budget tooling set up in CI.

Milestone 1 — Widget Runtime (Week 2-3)

Runtime agent completes Layer 3. Widget loads in sandbox. SDK works. Bridge protocol tested. 3 built-in widgets running.

Milestone 2 — Canvas MVP (Week 3-5)

Canvas Core agent completes 4A-1 (rendering, input, transforms). Canvas Tools agent begins 4A-2 (stickers, entities, hierarchy). Stickers placeable. Entities working. Save/load from Supabase. Slug-based URLs working. Canvas Wiring (4A-3) and Canvas Panels (4A-4) agents can start in parallel once 4A-1 is stable.

Milestone 3 — Social (Week 4-5, parallel)

Social agent completes Layer 1. Two users on same canvas see each other. Presence working. Realtime entity sync working.

Milestone 4 — Widget Lab (Week 5-7)

Lab agent completes Layer 2. Widget created in Lab loads in canvas. AI generation working. Publish pipeline to Marketplace stub.

Milestone 5 — Spatial + VR (Week 6-8, parallel with Lab)

Spatial agent completes Layer 4B. Canvas opens in Three.js mode. Quest 3 browser loads it. Hand tracking works for basic select and move.

Milestone 6 — Platform Shell (Week 7-9)

Shell agent completes Canvas Gallery, auth UI, settings. Full user flow: sign up → create canvas → share → collaborate.

Milestone 7 — Marketplace Beta (Week 9-11)

Market agent completes Layer 5. Widgets publishable and installable. Paid widget flow with Stripe. Creator dashboard live.

Milestone 8 — Protocol & OS Layer (Week 10-12)

Protocol spec published. npm packages published. Widget validator live. StickerNest is now a platform others can build on.

Milestone 9 — Web & Mobile Polish (Week 8-10, parallel)

Web desktop experience polished: all keyboard shortcuts, multi-panel layouts, drag-and-drop, and browser compatibility verified across Chrome, Firefox, Safari, and Edge. Mobile layout complete. Canvas gallery and canvas editor fully usable

on phone and tablet. Touch input system tested on iOS Safari and Android Chrome. PWA installable. Both web and mobile testing gates pass.

Milestone 10 — Launch Readiness (Week 11-13)

Onboarding flow complete. Billing and subscription tiers live with Stripe. Notification system working. Version history working. Search working. Import/export working. a11y audit passed. Performance budgets green in CI. Sentry error tracking live. All E2E tests passing.

What a "Done" V5 Looks Like

- User signs up in their desktop browser, lands on Canvas Gallery
- Creates a canvas, sets a slug (`stickernest.app/u/kimber/morning`)
- Places stickers, wires pipelines, and builds widgets using the full desktop editor with keyboard shortcuts and multi-panel layout
- Opens Widget Lab, uses AI generation to build a custom widget, previews it live, publishes to Marketplace
- Attaches a Google Calendar widget to a calendar sticker
- Invites a friend — friend sees their cursor in real time
- Types `/` to open the AI command bar, says "set all timers to 25 minutes"
- Later, opens the same canvas on their phone — canvas gallery loads fast, stickers are tappable, pinch-to-zoom works perfectly
- Opens the same canvas in Quest 3 — hand-tracks to interact with widgets in VR
- Another user buys the published widget, installs it, wires it to their canvas via a pipeline
- The canvas is listed as a SaaS product — others subscribe to use it
- All of this runs on the open StickerNest protocol
- The whole experience is accessible, fast, works offline, and feels native on every device — desktop browser, phone, tablet, and VR headset